

Introduction

This testing plan outlines the strategy, scope and methodologies for the secure QR authentication including the administration dashboard, mobile application and backend services. The objective is to verify that the entire system operates properly across all essential user and admin features which includes mobile QR code scanning, telemetry data gathering, admin management workflows and incident reporting while at the same time maintaining strong security measures like Row-Level Security, JWT token verification and the mandatory transport-layer security.

Testing scope

In-Scope:

- **Database and Row-Level Security (RLS) Policies:** Validation of role-specific data isolation (Admin vs standard user access on User, QrCode, Alert, ScanLog and ActivityLog)
- **Authorization and authentication:** Generation of JWT token, signature authentication, expiration verifications and password encryption limitations.
- **Transport-Layer Security (HTTPS):** Confirm that all communications from client to server, authentication data and API requests are enforced to happen exclusively over HTTPS to avoid man-in-the-middle or sniffing.
- **Core workflow:** Generating QR codes, scanning/telemetry recording and incident report submissions.
- **Data integrity:** Primary keys, referential integrity of foreign keys and domain restrictions (e.g., timestamps, legitimate URLs, and default statuses).

Out-of-Scope:

- Performance of third-party infrastructure or uptime external hosting providers.

Testing types and methodologies

Unit Testing: Testing constraint rules such as whether registered email can be registered again or password has a minimum of 8 characters.

Security & Access Control Testing: Testing of HTTPS enforcement and use of JWT. Testing if users have unauthorized access to admin dashboard

Integration Testing: Verify end-to-end workflows, such as scanning a generated QR code successfully with the mobile application, generating a record in ScanLog, or submitting an incident report linking properly to QRCode and User.

Resource Requirements

Supabase and Render to be up and running
Mobile application, VAFPQR, to be in its beta version.
Supabase to have all RLS policies and data types to be finalised
User interfaces of admin dashboard and mobile app to be completed
Frontends are connected to the backends of the system.

Test environment

Database Management System: PostgreSQL (Supabase)

Backend Server: Render

Execution Tools: Frontend mobile application and admin dashboard web interfaces.

Pass/Fail criteria

Pass: All critical security constraints are met at 100%, HTTPS enforcement and RLS policies effectively prevent any unauthorized access, all foreign key relationships preserve referential integrity without any unreferenced records and valid user workflows finish successfully.

Fail: Any occurrence where a standard user can bypass RLS to access or alter restricted tables, unsecured communication taking place, database allowing invalid plaintext data entries or the outcome of the test case does not align with the expected result.